

Stock Wave Project

Malak Tarek	202206120
Judy Mostafa	202207017
Sohaila Ziad	202206614
Hana Ahmed	202206963

Dr. Mohamed Khalaf

Current Issues

Stock Market Analysis System (Using Streamlit)

◆ 1. Project Overview

This project is a web-based application developed using Streamlit that allows users to view and analyze stock market data. The system provides simple visualizations and basic stock information to help users understand market trends.

◆ 2. Objectives

- To understand how stock market data is collected and displayed
- To build a simple interactive web application
- To visualize stock price trends using charts
- To provide basic stock information to users

◆ 3. Scope

The system will:

- Allow users to search for stock symbols
- Display current and historical stock prices
- Show graphical representation of stock trends
- Provide a simple and user-friendly interface

◆ 4. Functional Requirements

The application should:

- Accept stock symbol input from the user
- Fetch stock data from an API (e.g., Yahoo Finance)
- Display:
 - Stock name and current price
 - Historical data (last 7 days / 1 month)
- Generate line charts for price trends

Handle invalid inputs with error messages

◆ 5. Non-Functional Requirements

- The system should be easy to use
- The interface should be simple and clear
- Data should load quickly
- The application should be reliable and responsive

◆ 6. Tools and Technologies

- **Frontend & Backend:** Streamlit ·
- **Programming Language:** Python ·
- **Libraries:**
 - Pandas (data handling) ○
 - Matplotlib or Plotly (visualization) ○
 - yfinance (stock data API)

◆ 7. System Workflow

1. User enters a stock symbol (e.g., AAPL)
2. The system retrieves stock data from the API
3. The data is processed and displayed
4. Charts are generated to show trends

◆ 8. User Interface Requirements

- Input field for stock symbol
- Display area for stock data
- Chart section for visualization
- Clean and simple layout

◆ 9. Testing Requirements

- Test with valid stock symbols
- Test with invalid inputs
- Verify chart accuracy

- Check system performance

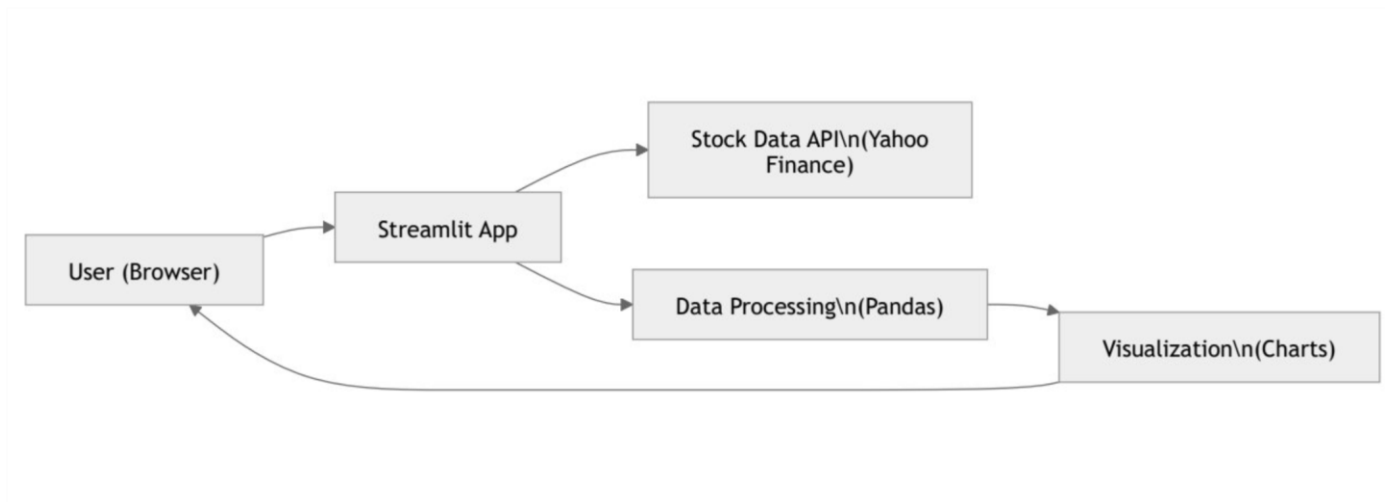
◆ 10. Expected Output

- Display of stock price information
- Graph showing stock trends
- Simple and interactive web interface

◆ 11. Conclusion

This project demonstrates basic knowledge of stock market data analysis and web application development using Streamlit. It is suitable for beginners and provides a foundation for more advanced systems.

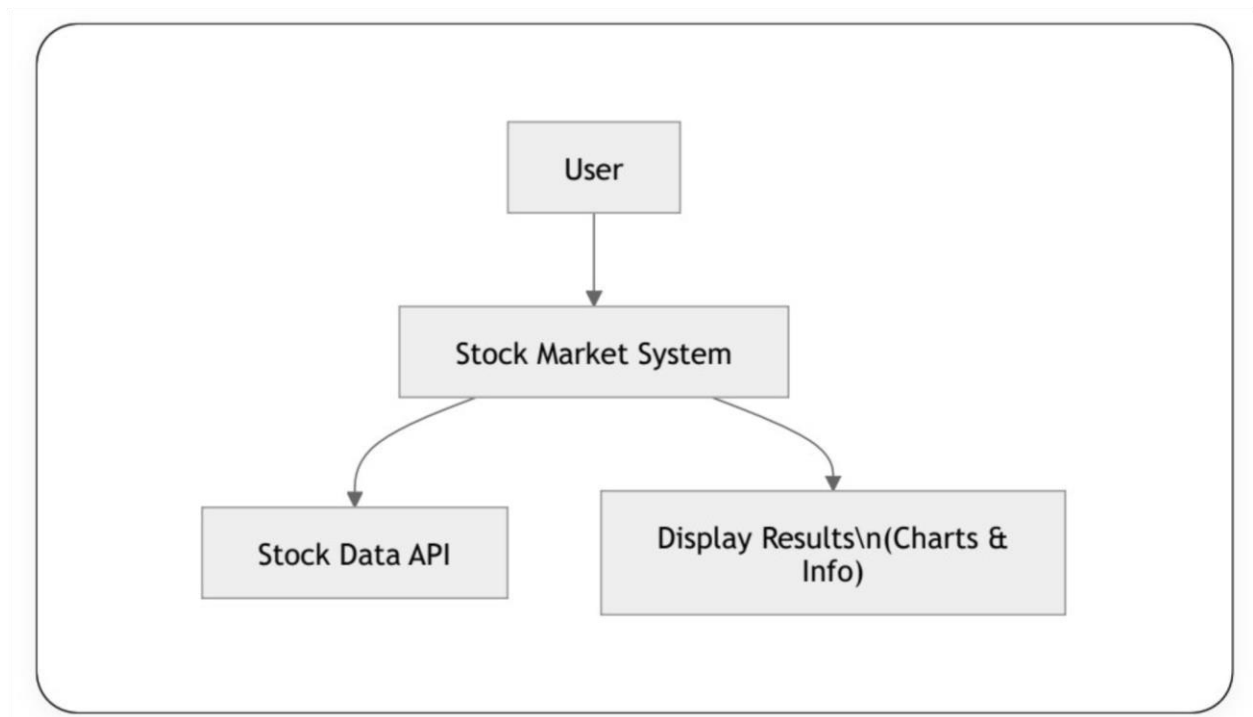
1. System Architecture Diagram (ImageStyle)



System Architecture Diagram Description

The diagram shows how the Stock Market Analysis System works. The user enters a stock symbol through the Streamlit web application. The system then retrieves stock data from the Yahoo Finance API, processes the data using Pandas, and generates charts for visualization. Finally, the stock information and graphs are displayed to the user in the browser through the Streamlit interface.

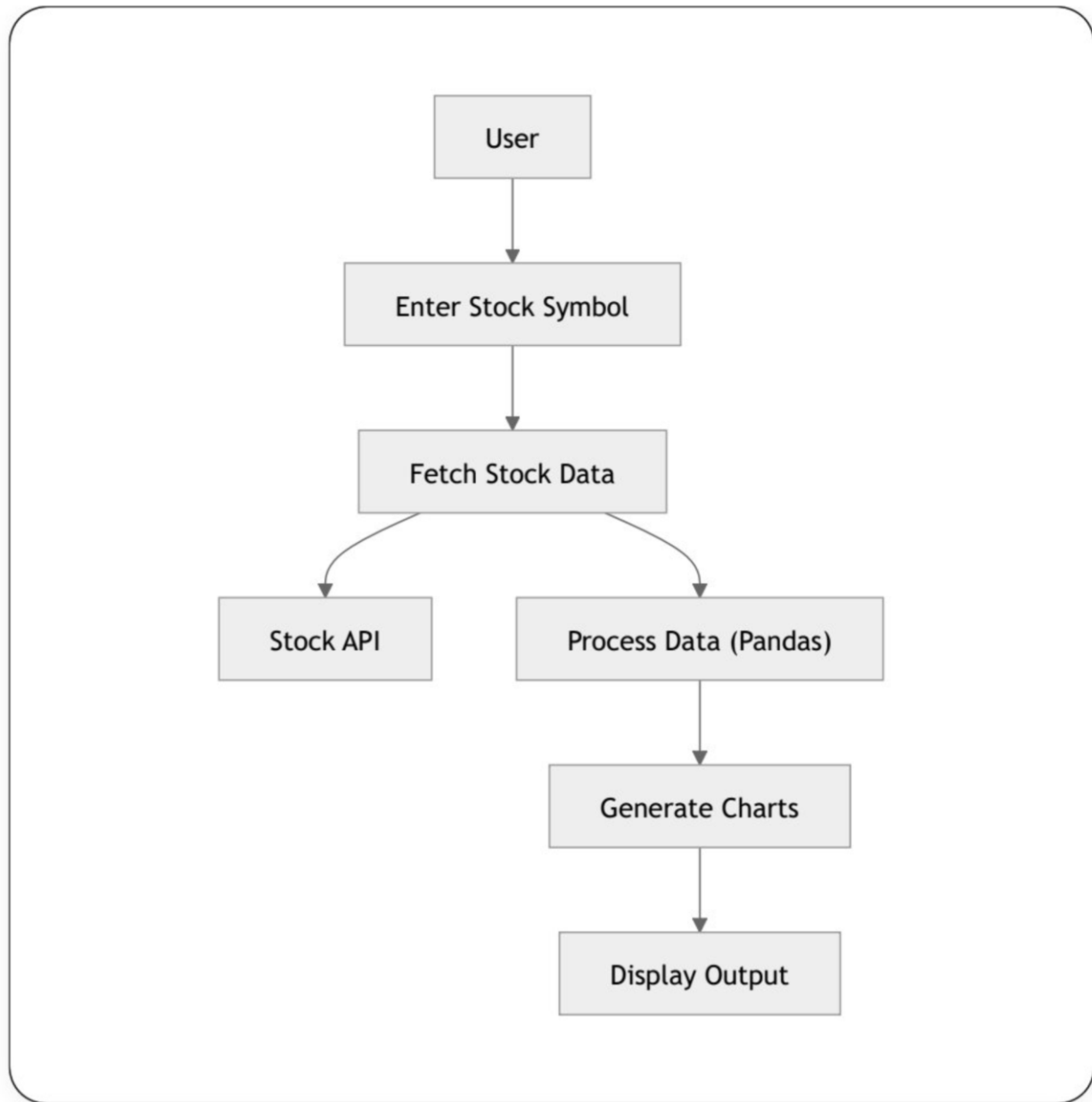
2. Data Flow Diagram (DFD Level 0)



Data Flow Diagram (DFD Level 0) Description

The DFD Level 0 diagram shows the basic flow of data in the Stock Market Analysis System. The user interacts with the system by entering a stock symbol request. The Stock Market System processes this request and communicates with the Stock Data API to retrieve stock information. After processing the data, the system displays the results to the user in the form of stock details, charts, and trend visualizations.

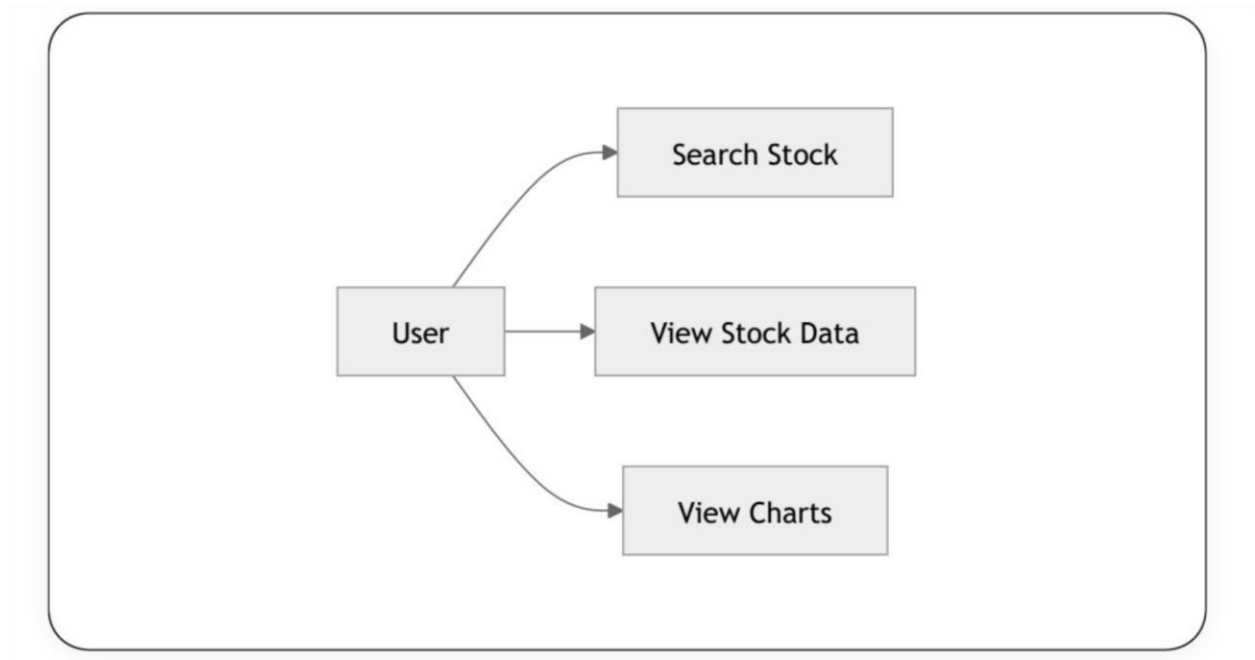
3. Data Flow Diagram (DFD Level 1)



Data Flow Diagram (DFD Level 1) Description

The DFD Level 1 diagram explains the detailed flow of data inside the Stock Market Analysis System. First, the user enters a stock symbol into the system. The system then fetches stock data by communicating with the Stock API. After retrieving the data, it is processed using Pandas for organization and analysis. The processed data is used to generate charts and visualizations, and finally the output is displayed to the user in the form of stock information and graphical trends.

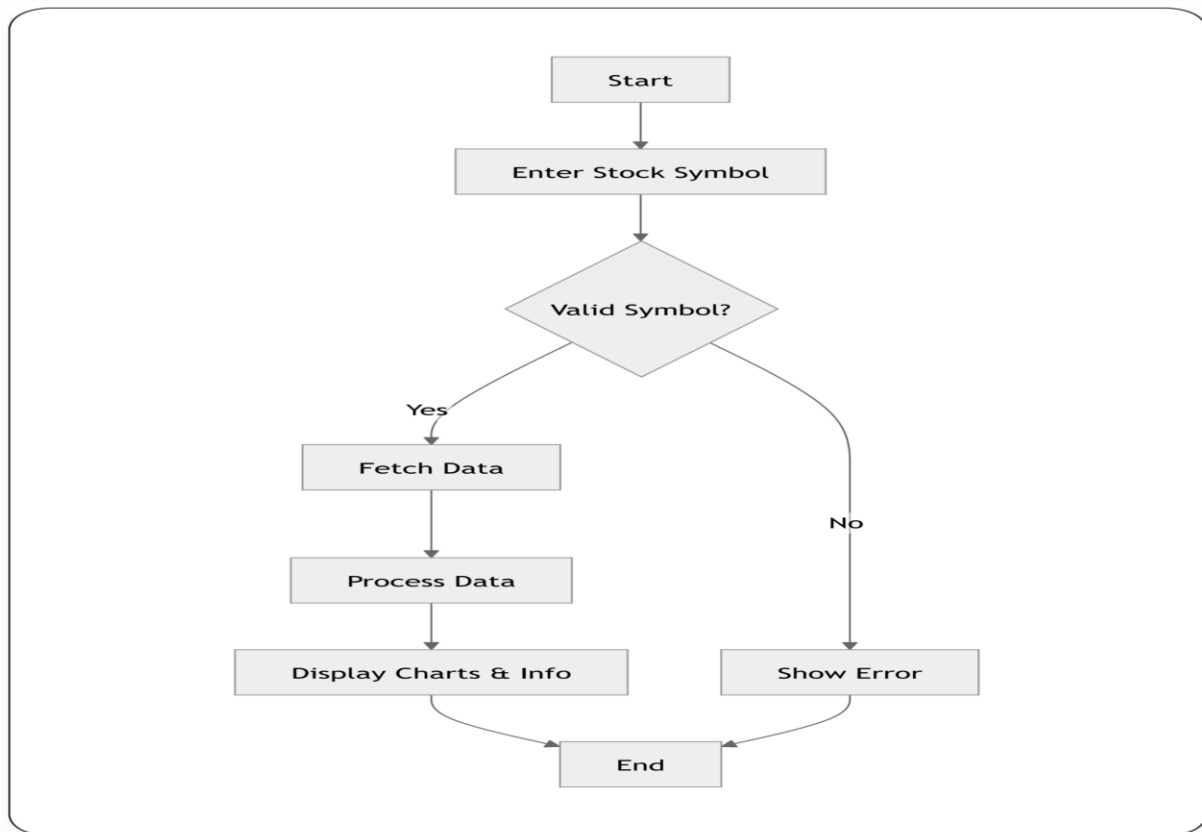
4. Use Case Diagram



Use Case Diagram Description

The Use Case Diagram shows the interactions between the user and the Stock Market Analysis System. The user can search for a stock by entering its symbol, view stock data such as prices and information, and view charts that display stock market trends and visualizations. These functions represent the main features provided by the system.

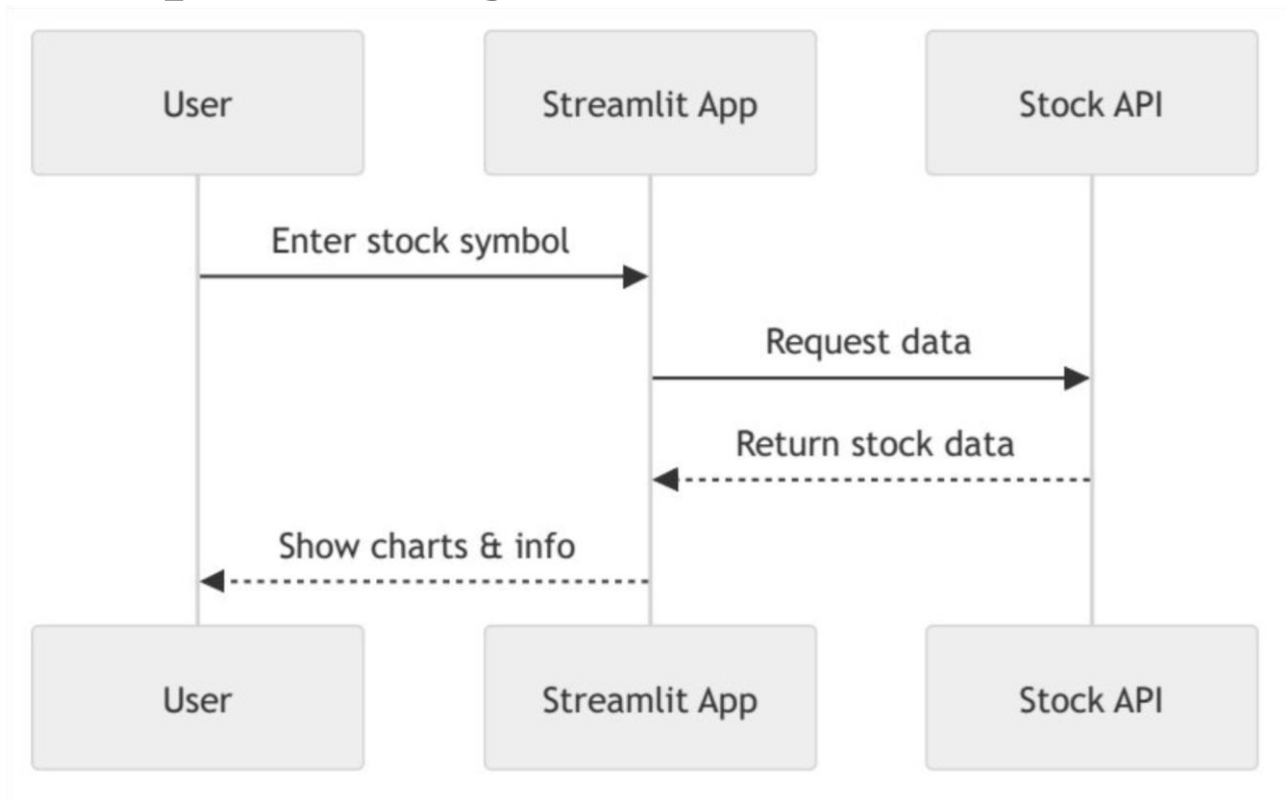
5. Activity Diagram



Activity Diagram Description

The Activity Diagram illustrates the workflow of the Stock Market Analysis System. The process starts when the user enters a stock symbol. The system then checks whether the symbol is valid or not. If the symbol is valid, the system fetches the stock data, processes it, and displays charts and stock information to the user. If the symbol is invalid, the system shows an error message. Finally, the process ends.

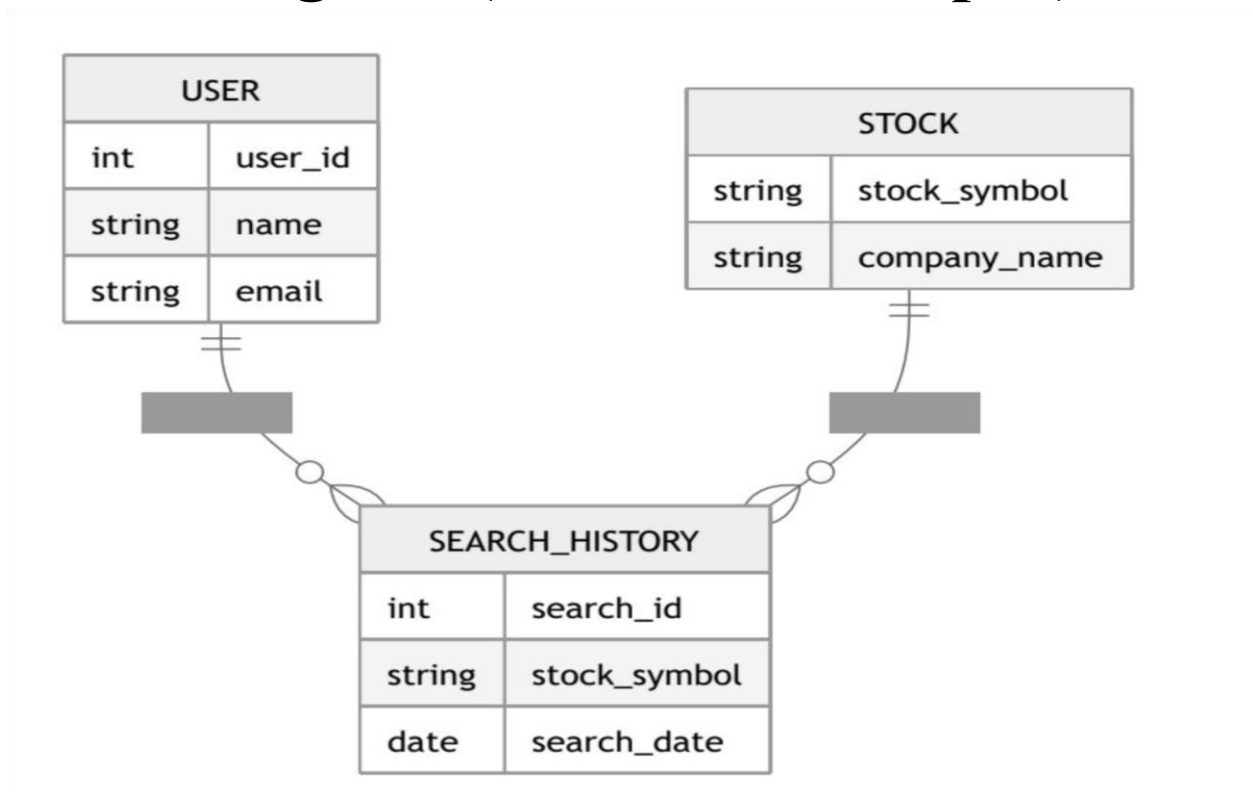
6. Sequence Diagram



Sequence Diagram Description

The Sequence Diagram shows the interaction between the User, the Streamlit Application, and the Stock API over time. First, the user enters a stock symbol into the Streamlit application. The application then sends a request to the Stock API to retrieve stock data. After the API returns the stock information, the Streamlit app processes the data and displays charts and stock information back to the user.

7. ER Diagram (Database – Simple)



ER Diagram Description

The ER Diagram represents the simple database structure of the Stock Market Analysis System. It contains three main entities: User, Stock, and Search_History. The User entity stores user information such as user ID, name, and email. The Stock entity stores stock details including stock symbol and company name. The Search_History entity records the stocks searched by users along with the search date. The relationships show that a user can search for multiple stocks, and each search is stored in the search history.

Screenshots

Deploy



Stock Market Analysis System

Enter a stock symbol to view stock data and price trends.

Enter Stock Symbol

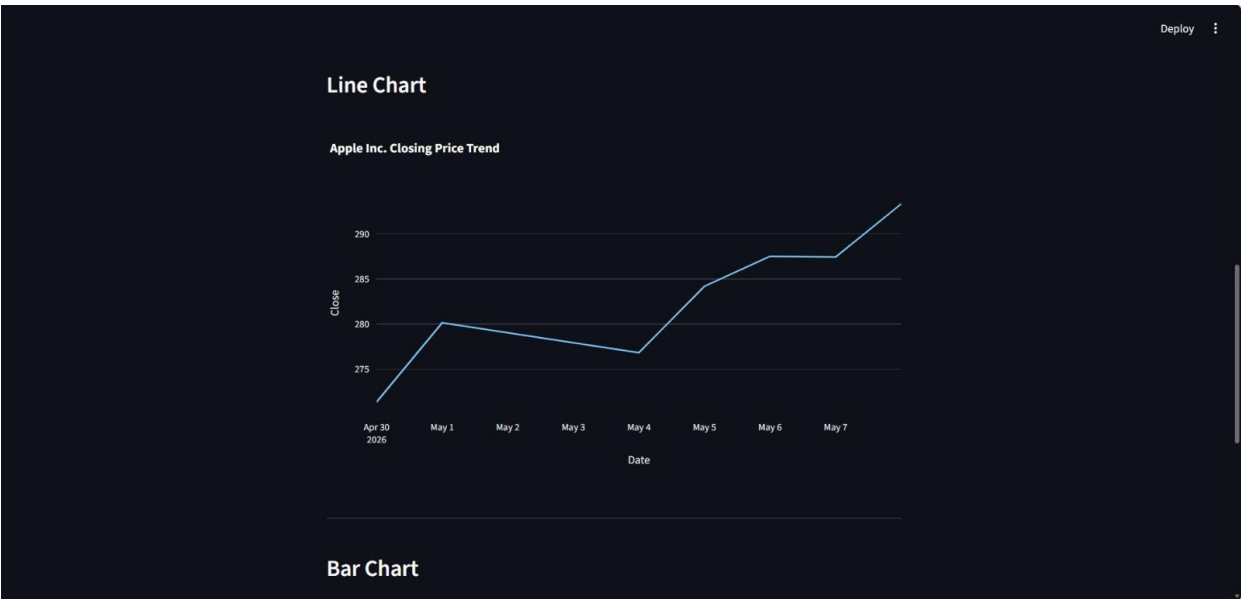
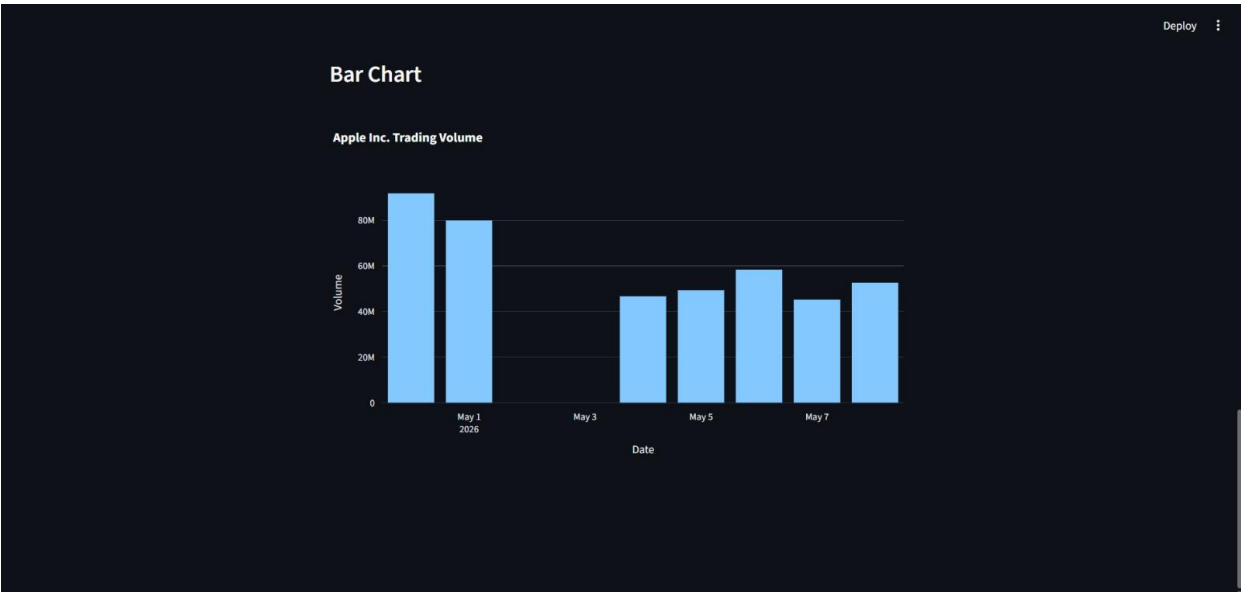
AAAAAA

Choose Period

7d

Search

Invalid stock symbol or no data found.



Historical Stock Data

Date	Open	High	Low	Close	Volume	Dividends	Stock Splits
2026-04-30 00:00:00-04:00	270.5	276	268.14	271.35	91848200	0	0
2026-05-01 00:00:00-04:00	278.86	287.22	278.37	280.14	79915400	0	0
2026-05-04 00:00:00-04:00	279.66	280.63	274.86	276.83	46668400	0	0
2026-05-05 00:00:00-04:00	276.93	284.57	276.5	284.18	49311700	0	0
2026-05-06 00:00:00-04:00	281.92	288.03	281.07	287.51	58336100	0	0
2026-05-07 00:00:00-04:00	289.27	292.13	285.78	287.44	45224300	0	0
2026-05-08 00:00:00-04:00	290.01	294.76	290	293.32	52631200	0	0

Line Chart

Apple Inc. Closing Price Trend



Stock Market Analysis System

Enter a stock symbol to view stock data and price trends.

Enter Stock Symbol

AAPL

Choose Period

7d

Search

Company Name: Apple Inc.

Current Price

\$293.32

High Price

\$294.76

Low Price

\$268.14

Volume

423,935,...

Historical Stock Data

Team Task Allocation

Member 1 – Project Lead / Backend Developer

Main Role: API Integration + Core Logic

- Set up Python project structure
- Integrate stock data API using **yFinance**
- Handle stock symbol input and validation
- Fetch current + historical stock data
- Manage error handling (invalid symbols, API failures)

Member 2 – Data Processing & Analysis

Main Role: Data Handling

- Clean and structure stock data using **Pandas**
- Extract required fields (open, close, high, low, volume)
- Prepare data for visualization
- Compute simple indicators (optional): moving average, daily change

Member 3 – Visualization Developer

Main Role: Charts & Graphs

- Create stock trend charts using **Matplotlib or Plotly**
- Build interactive visualizations (line charts, trends)
- Design graph layout for clarity
- Ensure charts update dynamically based on user input

Member 4 – UI Developer / Tester / Documentation

Main Role: Streamlit UI + Testing

- Build Streamlit interface (input box, buttons, layout)
- Display stock info and charts in UI
- Improve user experience (clean design)
- Test application with valid/invalid inputs
- Prepare documentation (report, screenshots, PPT if needed)